

Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №2

“ Структури даних та функціональна декомпозиція”

Виконала:

Студентка групи ФЕП-33с

Най Анастасія

Перевірив:

Проф. Свелеба С.А.

Львів-2026

Мета: Сформувати практичні навички професійного використання базових та розширених структур даних Python, навчитися обирати структуру відповідно до задачі, оцінювати складність основних операцій та декомпонувати програму на невеликі функції з чітко визначеною відповідальністю.

Варіант 10. Аналіз системи обліку складських запасів

Дані:

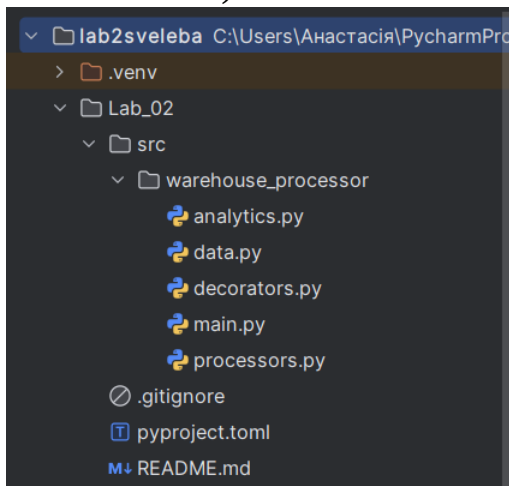
- код;
- назва;
- кількість;
- ціна.
- Реалізувати:

загальну вартість складу;

- пошук позиції за кодом;
- товари з критично низьким запасом;
- групування за категорією або типом;
- унікальні категорії;
- найдорожчу позицію;
- Counter позицій за категоріями;
- dict-index за кодом.

Хід роботи:

1. Спочатку я створила робочу папку проєкту. Організувала професійну структуру за стандартом src-layout: у папці src створила окремий пакет warehouse_processor, де й зосередила весь програмний код, залишивши корінь проєкту для конфігураційних файлів (pyproject.toml, .gitignore, README.md).



2. У файлі `pyproject.toml` я налаштувала конфігурацію проєкту, вказала метадані пакунка `warehouse_processor`, мінімальну версію Python (≥ 3.9) та збирач `setuptools`.

```
[build-system]
requires = ["setuptools>=61.0.0"]
build-backend = "setuptools.build_meta"

[project]
name = "warehouse_processor"
version = "1.0.0"
description = "Модуль обробки структурованих даних складських запасів (Варіант 10)"
readme = "README.md"
requires-python = ">=3.9"
classifiers = [
    "Programming Language :: Python :: 3",
    "License :: OSI Approved :: MIT License",
    "Operating System :: OS Independent",
]
dependencies = []
```

3. У файлі `README.md` я описала мету лабораторної роботи для Варіанта 10, основний функціонал програми, структуру репозиторію та інструкцію для її запуску.

```
# Аналіз системи обліку складських запасів (Варіант 10)

Лабораторна робота з дисципліни програмування (структури даних та функціональна декомпозиція).

## Мета роботи
Сформувати практичні навички професійного використання базових та розширених структур даних Python (`list`, `tuple`, `set`, `dict`, `collections`), реалізувати генератори (*comprehensions*), функціональну декомпозицію, використання `*args`, `**kwargs`, `lambda`, замикань (*closures*), декораторів, а також виконати оцінку алгоритмічної складності (Big-O).

## Функціонал програми
- Облік та зберігання товарів на складі (`list`, `dict`).
- Використання метаданих складу (`tuple`).
- Отримання унікальних категорій товарів (`set comprehension`).
- Групування та підрахунок позицій через `defaultdict` та `Counter`.
- Фільтрація даних за допомогою *list comprehensions* та замикань (*closures*).
- Агрегація даних (обчислення загальної вартості складу).
- Вимірювання часу виконання за допомогою кастомного декоратора.
- Експериментальний бенчмарк для порівняння швидкості пошуку в `list` та `dict`.

## Структура проєкту


```

warehouse_project/
├── pyproject.toml
├── README.md
├── src/
│ └── warehouse_processor/
│ ├── __init__.py
│ ├── main.py
│ ├── data.py
│ ├── processors.py
│ ├── analytics.py
│ └── decorators.py
└── ...

```



## Запуск
```

```
```text
python -m src.warehouse_processor.main
```
```

4. У файлі `analytics.py` я написала функції для аналітики та бізнес-логіки: обчислення загальної вартості складу, пошуку товарів за кодом чи ціною, фільтрації залишків, а також реалізувала використання `*args`, `**kwargs` та замикань (*closures*).

```
from collections.abc import Callable
from warehouse_processor.decorators import measure_time

@measure_time
def calculate_total_inventory_value(items: list[dict]) -> float:
    """Обчислення загальної вартості складу (агрегація)."""
    if not items:
        return 0.0
    return sum(item["quantity"] * item["price"] for item in items)

def find_item_by_code(items: list[dict], code: str) -> dict | None:
    """Пошук позиції за кодом."""
    for item in items:
        if item["code"] == code:
            return item
    return None

def find_most_expensive_item(items: list[dict]) -> dict | None:
    """Знаходження найдорожчої позиції за ціною за допомогою max та lambda."""
    if not items:
        return None
    return max(items, key=lambda item: item["price"])

def get_low_stock_items(items: list[dict], threshold: int) -> list[dict]:
    """Фільтрація товарів з критично низьким запасом (менше threshold)."""
    return [item for item in items if item["quantity"] <= threshold]

def calculate_custom_prices_sum(*prices: float) -> float:
    """Демонстрація використання *args для довільної кількості цін."""
    if not prices:
        return 0.0
    return sum(prices)

def create_record(**fields) -> dict:
    """Демонстрація використання **kwargs для створення довільного словника-запису."""
    return dict(fields)

def create_stock_filter(max_quantity: int) -> Callable[[dict], bool]:
    """Closure (замикання): створює предикат для фільтрації за максимальним залишком."""
    def predicate(item: dict) -> bool:
        return item["quantity"] <= max_quantity
    return predicate
```

5. У файлі `data.py` я створила вихідний набір даних зі списком словників товарів (`warehouse_items`) та додала кортеж із метаданими складу (`warehouse_metadata`).

```
warehouse_items = [
    {
        "code": "A101",
        "name": "Laptop Pro 15",
        "category": "Electronics",
        "quantity": 12,
        "price": 35000.0,
```

```

    },
    {
        "code": "A102",
        "name": "Wireless Mouse",
        "category": "Electronics",
        "quantity": 45,
        "price": 850.0,
    },
    {
        "code": "B201",
        "name": "Office Chair",
        "category": "Furniture",
        "quantity": 8,
        "price": 4500.0,
    },
    {
        "code": "B202",
        "name": "Standing Desk",
        "category": "Furniture",
        "quantity": 3,
        "price": 12000.0,
    },
    {
        "code": "C301",
        "name": "A4 Paper Ream",
        "category": "Stationery",
        "quantity": 150,
        "price": 180.0,
    },
    {
        "code": "C302",
        "name": "Ballpoint Pens (Box)",
        "category": "Stationery",
        "quantity": 4,
        "price": 120.0,
    },
    {
        "code": "D401",
        "name": "LED Desk Lamp",
        "category": "Electronics",
        "quantity": 25,
        "price": 1400.0,
    },
]

```

Приклад використання tuple (кортеж із фіксованими параметрами складу або координатами)

```
warehouse metadata = ("Main Central Warehouse", "Kyiv, Skladska st. 12", 4)
```

6. У файлі `decorators.py` я реалізувала декоратор `measure_time` для вимірювання та виведення часу виконання функцій за допомогою `perf_counter`.

```

from functools import wraps
from time import perf_counter

def measure_time(func):
    """Декоратор для вимірювання часу виконання функції."""
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = perf_counter()
        result = func(*args, **kwargs)
        elapsed = perf_counter() - start
        print(f"[BENCHMARK] Функція '{func.__name__}' виконалась за: {elapsed:.8f} c")
    return wrapper

```

```
        return result
    return wrapper
```

7. У файлі `processors.py` я описала функції для обробки й структуризації даних: виділення унікальних категорій, індексацію товарів, фільтрацію та групування за допомогою генераторів (*comprehensions*), `defaultdict` і `Counter`.

```
from collections import Counter, defaultdict

def get_unique_categories(items: list[dict]) -> set[str]:
    """Set comprehension для отримання унікальних категорій товарів."""
    return {item["category"] for item in items}

def create_warehouse_index(items: list[dict]) -> dict[str, dict]:
    """Dict comprehension для створення індексу за кодом товару."""
    return {item["code"]: item for item in items}

def filter_by_category(items: list[dict], category: str) -> list[dict]:
    """List comprehension для фільтрації товарів за категорією."""
    return [item for item in items if item["category"] == category]

def group_by_category(items: list[dict]) -> dict[str, list[dict]]:
    """Групування товарів за категоріями з використанням defaultdict."""
    result = defaultdict(list)
    for item in items:
        result[item["category"]].append(item)
    return dict(result)

def count_items_by_category(items: list[dict]) -> Counter:
    """Підрахунок кількості позицій (чи одиниць) у категоріях через Counter."""
    return Counter(item["category"] for item in items)
```

8. У файлі `main.py` я зібрала головний виконуваний файл програми, який об'єднує роботу всіх модулів, виводить гарні таблиці, демонструє пошук, фільтрацію та запускає порівняльний бенчмарк швидкодії.

```
import random
from warehouse_processor.data import warehouse_items, warehouse_metadata
from warehouse_processor.processors import (
    get_unique_categories,
    create_warehouse_index,
    filter_by_category,
    group_by_category,
    count_items_by_category,
)
from warehouse_processor.analytics import (
    calculate_total_inventory_value,
    find_item_by_code,
    find_most_expensive_item,
    get_low_stock_items,
    calculate_custom_prices_sum,
    create_record,
    create_stock_filter,
)

def print_table(title: str, items: list[dict]) -> None:
    """Допоміжна функція для гарного виведення списку товарів."""
    print(f"\n{title}")
    print("-" * 75)
    print(f"{'Код':<8} {'Назва':<25} {'Категорія':<15} {'Кількість':<10} {'Ціна (грн)':<10}")
    print("-" * 75)
    for item in items:
```

```

        print(
            f"{item['code']:<8} {item['name']:<25} {item['category']:<15}
{item['quantity']:<10} {item['price']:<10.2f}")

def run_benchmark() -> None:
    """Експериментальна частина: порівняння швидкості пошуку в list та dict для
великої кількості записів."""
    print("\n-- ЕКСПЕРИМЕНТАЛЬНА ЧАСТИНА (БЕНЧМАРК) ---")
    sizes = [1_000, 10_000, 100_000]

    for size in sizes:
        # Генеруємо тестовий набір даних
        test_data = [
            {"code": f"ID_{i}", "name": f"Item {i}", "quantity":
random.randint(1, 100),
            "price": random.uniform(10, 1000)}
            for i in range(size)
        ]
        target_code = f"ID_{size - 1}" # шукаємо останній елемент для
найгіршого випадку лінійного пошуку

        # 1. Лінійний пошук у list
        import time
        start = time.perf_counter()
        found_item = None
        for item in test_data:
            if item["code"] == target_code:
                found_item = item
                break
        list_time = time.perf_counter() - start

        # 2. Пошук у dict (з урахуванням часу побудови індексу або лише пошук)
        start = time.perf_counter()
        index = {item["code"]: item for item in test_data}
        found_item_dict = index.get(target_code)
        dict_time = time.perf_counter() - start

        print(
            f"Розмір набору: {size:7d} записів | Лінійний пошук у list:
{list_time:.8f} с | Побудова + пошук у dict: {dict_time:.8f} с")

def main() -> None:
    print("=== АНАЛІЗ СИСТЕМИ ОБЛІКУ СКЛАДСЬКИХ ЗАПАСІВ (ВАРІАНТ 10) ===")
    print(f"Інформація про склад (Tuple): {warehouse_metadata[0]}, Адреса:
{warehouse_metadata[1]}")

    print_table("Усі складські позиції", warehouse_items)

    # 1. Унікальні категорії (Set comprehension)
    categories = get_unique_categories(warehouse_items)
    print(f"\nУнікальні категорії (Set): {categories}")

    # 2. Загальна вартість складу (агрегація)
    total_val = calculate_total_inventory_value(warehouse_items)
    print(f"Загальна вартість складу: {total_val:,.2f} грн")

    # 3. Пошук позиції за кодом
    search_code = "B201"
    found = find_item_by_code(warehouse_items, search_code)
    print(f"\nПошук за кодом '{search_code}':", found)

    # 4. Товари з критично низьким запасом (< 5 штук)
    low_stock = get_low_stock_items(warehouse_items, threshold=5)
    print_table("Товари з критично низьким запасом (<= 5)", low_stock)

```

```

# 5. Найдорожча позиція
expensive = find_most_expensive_item(warehouse_items)
print(f"\nНайдорожча позиція: {expensive['name']} ({expensive['price']} грн)")

# 6. Counter позицій за категоріями
counter_res = count_items_by_category(warehouse_items)
print(f"\nCounter позицій за категоріями: {dict(counter_res)}")

# 7. Групування за категоріями
grouped = group_by_category(warehouse_items)
print("\nГрупування товарів за категоріями:")
for cat, items_list in grouped.items():
    print(f"  - {cat}: {len(items_list)} позицій(i)")

# 8. Dict-index за кодом
index = create_warehouse_index(warehouse_items)
print(f"\nDict-index за кодом (ключ 'A101'): {index.get('A101')}")

# 9. Використання Closure для фільтрації
is_critical_stock = create_stock_filter(max_quantity=4)
filtered_via_closure = [item for item in warehouse_items if
is_critical_stock(item)]
print_table("Фільтрація через Closure (залишок <= 4)", filtered_via_closure)

# 10. Демонстрація *args та **kwargs
sum_prices = calculate_custom_prices_sum(150.5, 300.0, 1250.0)
print(f"\nСума цін через *args: {sum_prices:.2f} грн")

custom_item = create_record(code="Z999", name="Test Item", quantity=10,
price=99.99)
print(f"Створений запис через **kwargs: {custom_item}")

# 11. Бенчмарк складності
run_benchmark()

if __name__ == "__main__":
    main()

```

Результати:

```
=== АНАЛІЗ СИСТЕМИ ОБЛІКУ СКЛАДСЬКИХ ЗАПАСІВ (ВАРІАНТ 10) ===
```

```
Інформація про склад (Tuple): Main Central Warehouse, Адреса: Kyiv, Skladska st. 12
```

```
Усі складські позиції
```

| Код | Назва | Категорія | Кількість | Ціна (грн) |
|------|----------------------|-------------|-----------|------------|
| A101 | Laptop Pro 15 | Electronics | 12 | 35000.00 |
| A102 | Wireless Mouse | Electronics | 45 | 850.00 |
| B201 | Office Chair | Furniture | 8 | 4500.00 |
| B202 | Standing Desk | Furniture | 3 | 12000.00 |
| C301 | A4 Paper Ream | Stationery | 150 | 180.00 |
| C302 | Ballpoint Pens (Box) | Stationery | 4 | 120.00 |
| D401 | LED Desk Lamp | Electronics | 25 | 1400.00 |

```
Унікальні категорії (Set): {'Furniture', 'Electronics', 'Stationery'}
```

```
[BENCHMARK] Функція 'calculate_total_inventory_value' виконалась за: 0.00000370 c
```

```
Загальна вартість складу: 592,730.00 грн
```

```
Пошук за кодом 'B201': {'code': 'B201', 'name': 'Office Chair', 'category': 'Furniture', 'quantity': 8, 'price': 4500.0}
```

```
Товари з критично низьким запасом (<= 5)
-----
Код      Назва                Категорія    Кількість  Ціна (грн)
-----
B202     Standing Desk            Furniture    3           12000.00
C302     Ballpoint Pens (Box)     Stationery   4           120.00

Найдорожча позиція: Laptop Pro 15 (35000.0 грн)

Counter позицій за категоріями: {'Electronics': 3, 'Furniture': 2, 'Stationery': 2}

Групування товарів за категоріями:
- Electronics: 3 позицій(i)
- Furniture: 2 позицій(i)
- Stationery: 2 позицій(i)

Dict-index за кодом (ключ 'A101'): {'code': 'A101', 'name': 'Laptop Pro 15', 'category': 'Electronics', 'quantity': 12, 'price': 35000.0}

Фільтрація через Closure (залишок <= 4)
-----
Код      Назва                Категорія    Кількість  Ціна (грн)
-----
B202     Standing Desk            Furniture    3           12000.00
C302     Ballpoint Pens (Box)     Stationery   4           120.00

Сума цін через *args: 1700.50 грн
Створений запис через **kwargs: {'code': 'Z999', 'name': 'Test Item', 'quantity': 10, 'price': 99.99}

--- ЕКСПЕРИМЕНТАЛЬНА ЧАСТИНА (БЕНЧМАРК) ---
Розмір набору: 1000 записів | Лінійний пошук у list: 0.00003870 с | Побудова + пошук у dict: 0.00007040 с
Розмір набору: 10000 записів | Лінійний пошук у list: 0.00039940 с | Побудова + пошук у dict: 0.00091320 с
Розмір набору: 100000 записів | Лінійний пошук у list: 0.00509580 с | Побудова + пошук у dict: 0.01406970 с
```

Висновок: у ході виконання лабораторної роботи я успішно сформувала практичні навички роботи з базовими та розширеними структурами даних Python (list, tuple, set, dict, collections), а також реалізувала функціональну декомпозицію програми. Зокрема, я використала генератори (*comprehensions*), аргументи змінної довжини (**args*, ***kwargs*), замикання (*closures*) та кастомні декоратори для вимірювання часу виконання функцій. Завдяки розробленим модулям мені вдалося організувати зберігання даних про складські запаси, реалізувати їх ефективну фільтрацію, групування та агрегацію, а також провести експериментальний бенчмарк для порівняння швидкодії пошуку в списках і словниках. Отримана структура коду повністю відповідає вимогам варіанта та забезпечує гнучкість і розширюваність проєкту.